

Описание языка L_1

С.Ю. Скоробогатов

1 сентября 2009

1 Введение

Язык L_1 представляет собой игрушечный императивный язык программирования, предназначенный для проведения лабораторных работ по курсу «Конструирование компиляторов».

2 Лексика

В языке L_1 мы будем различать пять типов лексем: идентификаторы (2.2), ключевые слова (2.3), константы (2.4), знаки операций и другие разделители (2.5). Компилятор языка L_1 игнорирует пробелы, знаки табуляции и перевода строки, расположенные между лексемами. При этом требуется, чтобы хотя бы один из этих символов разделял смежные идентификаторы, ключевые слова и константы.

При разбиении входного потока на лексемы должно учитываться *правило самой длинной лексемы*, а именно: если входной поток был разбит на лексемы вплоть до некоторой позиции, то начиная с этой позиции из входного потока выбирается самая длинная последовательность символов, которая может составлять лексему.

2.1 Комментарии

Символ `*`, расположенный в начале строки программы (в первой позиции), означает, что вся строка является комментарием.

Символы `***`, расположенные в любом месте строки, означают, что после них до конца строки идёт комментарий.

2.2 Идентификаторы

Идентификатор представляет собой последовательность букв и цифр, начинающуюся с буквы. При этом к цифрам мы дополнительно отнесём знак подчёркивания. Реализации языка L_1 , поддерживающие Unicode, должны разрешать использование в идентификаторах букв любых алфавитов, охваченных стандартом Unicode.

В языке L_1 идентификаторы, начинающиеся с заглавной буквы, используются для именования функций и неизменяемых переменных, а идентификаторы, начинающиеся со строчной буквы – для именования изменяемых переменных.

2.3 Ключевые слова

Следующие идентификаторы зарезервированы для использования в качестве ключевых слов:

and	else	new	then
array	elsif	not	to
assert	end	NULL	while
bool	F	or	xor
char	if	return	
define	int	step	
do	mod	T	

2.4 Константы

Константы в языке L_1 представляют собой изображения значений, известных на момент компиляции, и бывают нескольких типов.

2.4.1 Целочисленные константы

Целочисленные константы в языке L_1 изображают неотрицательные целые числа

Код	Аббревиатура	Код	Аббревиатура	Код	Аббревиатура	Код	Аббревиатура
0	NUL	8	BS	16	DLE	24	CAN
1	SOH	9	TAB	17	DC1	25	EM
2	STX	10	LF	18	DC2	26	SUB
3	ETX	11	VT	19	DC3	27	ESC
4	EOT	12	FF	20	DC4	28	FS
5	ENQ	13	CR	21	NAK	29	GS
6	ACK	14	SO	22	SYN	30	RS
7	BEL	15	SI	23	ETB	31	US

Таблица 1: Аббревиатуры управляющих символов в символьных и строковых константах.

в диапазоне от 0 до 2147483647. Они записываются в виде последовательностей цифр, перед которыми в фигурных скобках могут указываться основания систем счисления.

Цифрами считаются арабские цифры от 0 до 9, а также латинские буквы. При этом А обозначает 10, В обозначает 11, и т.д.

Основание системы счисления записывается в виде десятичного числа в диапазоне от 2 до 36. Если основание не указано, то целочисленная константа считается десятичной.

Целочисленная константа не может содержать пробелов, знаков табуляции и перевода строки.

Примеры целочисленных констант:

```
12000000      {16} 7FFFF
{2} 100111011 {20} gh10fj
{03} 1202211  0
```

2.4.2 Символьные константы

Символьные константы изображают символы Unicode или ASCII в зависимости от того, поддерживает компилятор языка L_1 стандарт Unicode или не поддерживает.

Символы, которые не являются управляющими, записываются в апострофах, причём для изображения символа «апостроф» используется пара апострофов:

```
'Q'  'я'  '1'  '&'  ''
```

Управляющие символы ASCII записываются как знак #, за которым следует аббревиатура символа (см. таблицу 1). Кроме того, любой символ с кодом x может быть представлен как $\#\{x_{16}\}$, где x_{16} – шестнадцатеричная запись числа x . Примеры:

```
#CR *** перевод каретки
#{D} *** тоже перевод каретки
```

2.4.3 Строковые константы

Строковая константа в языке L_1 представляет собой последовательность так называемых *строковых секций*, каждая из которых описывает цепочку символов Unicode или ASCII. Строковые секции в программе должны быть разделены пробелами, знаками табуляции или перевода строки.

Основная форма строковой секции выглядит как последовательность символов, заключённая в кавычки. При этом в эту последовательность не могут входить управляющие символы и кавычки.

Для представления кавычек используется специальная строковая секция, изображаемая как \$QUOT.

Каждый управляющий символ ASCII, входящий в строку, оформляется в виде отдельной строковой секции, которая записывается как знак \$, за которым следует аббревиатура символа (см. таблицу 1). Кроме того, любой символ с кодом x может быть представлен как отдельная секция $\$\{x_{16}\}$, где x_{16} – шестнадцатеричная запись числа x .

Например, строковая константа

```
"We say" $QUOT "Hello , World!"
$QUOT $LF
```

в языке C могла бы быть записана как:

```
"We say \"Hello , World!\"\\n"
```

2.4.4 Булевские константы

Для представления «истины» используется константа **T**, а для представления «лжи» – константа **F**.

2.4.5 Ссылочная константа NULL

Ключевое слово **NULL** обозначает нулевую ссылку.

2.5 Знаки операций и другие разделители

В языке L_1 используются следующие знаки операций и разделители:

;	,			(	)
:=	**	*	/	-	+
=	<>	<	>	<=	>=

3 Синтаксис и семантика

Программа на языке L_1 представляет собой набор *определений функций* (3.1). Порядок определений не важен, то есть из функции A может быть вызвана функция B , даже если определение B расположено в тексте программы после определения A .

3.1 Определения функций

Определение функции состоит из *заголовка* и *тела*.

Заголовок функции начинается с ключевого слова **define**, за которым следует *тип* (3.2) возвращаемого функцией значения (он не указывается, если функция не возвращает значение), имя функции и *список формальных параметров* функции. Имя функции должно начинаться с заглавной буквы.

Список формальных параметров представляет собой заключённую в круглые скобки последовательность операторов-объявлений (3.4.1), возможно пустую. Операторы-объявления в последовательности разделяются запятыми.

Тело функции непосредственно следует за заголовком. Оно представляет собой последовательность *операторов* (3.4), разделён-

ных точками с запятой, и завершается ключевым словом **end**.

Пример определения функции:

```
define int array SumVectors
  (int array A, int array B)
  int size := Length(A);
  assert size = Length(B);
  int array C := new int[size];
  int i := 0 to size-1 do
    C[i] := A[i] + B[i]
  end;
  return C
end
```

Главная функция, на которую передаётся управление при запуске программы, всегда называется **Main**, получает массив строк в качестве параметра и возвращает целое число:

```
define int Main
  (char array array args)
  *** какие-то действия
  return 0
end
```

3.2 Типы данных

Язык L_1 является языком со строгой преимущественно статической проверкой. Это означает, что во время компиляции известен тип каждой локальной переменной, тип каждого параметра функции, а также типы возвращаемых функциями значений.

В языке определены три примитивных типа **int**, **char** и **bool**, обозначающие целые числа, символы Unicode (или ASCII) и булевские значения, соответственно. Значения примитивных типов могут располагаться во фреймах функций и в элементах массивов.

Переменным типа **int** можно присваивать значения типа **int** или **char**. Переменным типа **char** можно присваивать только значения типа **char**, а переменным типа **bool** можно присваивать только значения типа **bool**.

Кроме примитивных типов в программах на языке L_1 можно использовать ссылочные типы-массивы. Значения этих типов могут располагаться во фреймах функций и в элементах массивов. Они являются указателя-

ми на массивы, расположенные куче. Память под массивы выделяется с помощью операции **new** и автоматически освобождается сборщиком мусора.

Тип-массив задаётся с помощью ключевого слова **array**, которое записывается после изображения типа элементов массива.

Например, тип-массив с целыми элементами записывается как **int array**, а тип-массив, элементами которого являются ссылки на массивы символов, записывается как **char array array**.

Отметим, что переменные типа **char array** играют роль строк, и им можно присваивать строковые константы.

Переменной типа **T array**, где **T** – некоторый тип, можно присваивать либо ссылки на массивы, элементы которых имеют тип **T**, либо константу **NULL**.

3.3 Выражения

Выражения в языке L_1 формируются из символов операций, констант, имён переменных, параметров и функций, изображений типов и круглых скобок.

Все операции перечислены в таблице 2. Во второй колонке таблицы метaperеменные x и y обозначают подвыражения, метaperеменная f обозначает имя функции, а метaperеменная T – изображение типа. Порядок выполнения операций может быть изменён с помощью круглых скобок.

В операции вызова функции $f(x_1, \dots, x_n)$ типы фактических параметров должны совпадать с типами формальных параметров, заданными при объявлении функции. Кроме того, функция f должна возвращать значение.

В операции выделения памяти **new T[x]** размер x должен иметь тип **int**.

Операции сравнения **=** и **<>** позволяют сравнивать ссылки на массивы с константой **NULL**.

Для остальных операций в таблице 3 приведены зависимости типов возвращаемых значений от допустимых типов операндов.

3.4 Операторы

В языке L_1 операторы задают действия, выполняемые программой. Всего предусмотрено девять видов операторов: оператор-объявление (3.4.1), оператор присваивания (3.4.2), оператор вызова функции (3.4.3), оператор выбора (3.4.4), два цикла с предусловием (3.4.5), цикл с постусловием (3.4.6), оператор завершения функции (3.4.7) и оператор-предупреждение (3.4.8).

3.4.1 Оператор-объявление

Оператор-объявление служит для объявления и инициализации переменных. Он начинается с изображения типа, за которым следует список объявляемых переменных. Переменные в списке разделяются запятыми. Кроме того, сразу после имени переменной может быть указан знак **:=**, за которым должно следовать выражение, значением которого переменная инициализируется.

Примеры операторов-объявлений:

```
char c ;
int array A := new int [ 5 ] ,
           B := new int [ 10 ] ;
int x , y := B [ 3 ] , z := y
```

3.4.2 Оператор присваивания

Оператор присваивания служит для присваивания значений *ячейкам*. Под ячейками мы будем понимать локальные переменные, параметры функций и элементы массивов.

Синтаксически оператор присваивания выглядит как два выражения, между которыми стоит знак **:=**. Подразумевается, что значение правого выражения присваивается ячейке, определяемой левым выражением.

Разумеется, не всякое выражение может стоять в левой части оператора присваивания. Выражение в левой части должно определять ячейку, и, кроме того, эта ячейка должна быть изменяемой. Например, следующие операторы присваивания записаны неправильно:

```
5 := a [ 1 ] ;
Func ( x ) := 10 ;
Q := 5
```

Уровень приоритета	Операция	Описание	Порядок выполнения
1	$x[y]$	Доступ к элементу массива x , имеющему индекс y .	$\longrightarrow$
	$f(x_1, \dots, x_n)$	Вызов функции f с передачей фактических параметров $x_1, \dots, x_n$.	
	new $T[x]$	Выделение памяти в куче для массива типа T array размера x .	
2	$-x$	Изменение знака числа x .	$\longleftarrow$
	not x	Логическое НЕ.	
3	$x ** y$	Возведение x в степень y .	$\longleftarrow$
4	$x * y$	Произведение x на y .	$\longrightarrow$
	x / y	Частное от деления x на y .	
	$x \bmod y$	Остаток от деления x на y .	
5	$x + y$	Сумма x и y .	$\longrightarrow$
	$x - y$	Разность x и y .	
6	$x = y$	Сравнение x и y .	$\longrightarrow$
	$x < > y$		
	$x < y$		
	$x > y$		
	$x \leq y$		
	$x \geq y$		
7	$x \text{ and } y$	Логическое И.	$\longrightarrow$
8	$x \text{ or } y$	Логическое ИЛИ.	$\longrightarrow$
	$x \text{ xor } y$	Логическое ИСКЛЮЧАЮЩЕЕ ИЛИ.	

Таблица 2: Операции, их приоритет и порядок выполнения.

Напомним, что локальные переменные и параметры, имена которых начинаются с заглавной буквы, являются неизменяемыми, и не могут, соответственно, указываться в левой части оператора присваивания.

Приведём примеры правильных операторов присваивания:

```
x := a*2 + 10;
A[5] := B[6]*10;
Func(x, y)[i+1] := 0
```

Здесь мы считаем, что в последнем операторе присваивания функция `Func` возвращает ссылку на массив целых чисел.

3.4.3 Оператор вызова функции

Оператор вызова функции позволяет вызывать функции, не возвращающие значе-

ния.

Он выглядит как

```
f(x1, ..., xn)
```

Здесь f – имя функции, а $x_1 \dots x_n$ – выражения, вычисляющие значения фактических параметров функции. Типы этих выражений должны совпадать с типами формальных параметров, указанными в заголовке функции.

3.4.4 Оператор выбора

Оператор выбора осуществляет передачу управления на разные участки кода в зависимости от того, истинно или ложно некоторое условие.

В языке L_1 самый простой вариант оператора выбора содержит только один участок

x	y	$x[y]$
T array	int	T
T array	char	T

(a) Индексирование массива.

x	$-x$
int	int
char	int

(b) Унарный минус.

x	not x
bool	bool

(c) Логическое НЕ.

x	y	$x + y$
int	int	int
int	char	char
char	int	char

(d) Операция +.

x	y	$x - y$
int	int	int
char	char	int
char	int	char

(e) Операция −.

x	y	$x \text{ op } y$
int	int	int

(f) Операции **, *, /, mod.

x	y	$x \text{ op } y$
int	int	bool
int	char	bool
char	int	bool
char	char	bool
bool	bool	bool
T array	T array	bool

(g) Операции =, <>.

x	y	$x \text{ op } y$
int	int	bool
int	char	bool
char	int	bool
char	char	bool

(h) Операции <, >, <=, >=.

x	y	$x \text{ op } y$
bool	bool	bool

(i) Операции and, or, xor.

Таблица 3: Зависимости типа возвращаемого значения от типов операндов.

кода, на который передаётся управление в случае истинности условия:

```

if условие then
    операторы
end

```

Условие является выражением, вычисляющим булевское значение. Операторы, расположенные между ключевыми словами **then** и **end**, разделяются точками с запятой.

Во втором варианте оператора выбора указываются два участка кода:

```

if условие then
    операторы_1
else
    операторы_2
end

```

Третий вариант содержит произвольное количество условий и участков кода:

```

if условие_1 then
    операторы_1
elsif условие_2 then
    операторы_2
...
elsif условие_N then
    операторы_N

```

```

else
    операторы_N+1
end

```

Пример оператора выбора:

```

if a < 0 then
    sign := −1
elsif a = 0 then
    sign := 0
else
    sign := 1
end

```

3.4.5 Циклы с предусловием

Цикл с предусловием осуществляет повторное выполнение некоторого участка кода, называемого телом цикла, до тех пор, пока некоторое условие, проверяемое до выполнения тела цикла, остаётся истинным.

В языке L_1 присутствуют два варианта цикла с предусловием.

Первый вариант выглядит следующим образом:

```

while условие do
    операторы
end

```

Условие является выражением, вычисляющим булевское значение. Операторы в теле цикла разделяются точкой с запятой.

Второй вариант цикла с предусловием является вариацией цикла `for`:

```
i := a to b step s do
  операторы
end
```

Эта запись подразумевает, что некоторая переменная i инициализируется значением выражения a и затем на каждой итерации цикла к i прибавляется значение выражения s до тех пор, пока i не примет значение b .

Шаг цикла s должен иметь тип `int`. Если шаг равен 1, то его можно не указывать.

Переменная i может иметь тип `int` или `char`. При этом предусмотрена возможность объявления переменной прямо в операторе цикла. Например:

```
char letter := 'A' to 'Z' do
  Print(letter)
end
```

3.4.6 Цикл с постусловием

Цикл с постусловием осуществляет повторное выполнение некоторого участка кода, называемого телом цикла, до тех пор, пока некоторое условие, которое первый раз проверяется после выполнения тела цикла, остаётся истинным.

В языке L_1 цикл с постусловием выглядит следующим образом:

```
do
  операторы
while условие
```

Условие является выражением, вычисляющим булевское значение. Операторы в теле цикла разделяются точкой с запятой.

3.4.7 Оператор завершения функции

Оператор завершения функции прекращает выполнение функции.

Оператор завершения может вызываться в любом месте тела функции. При этом для функции, возвращающей значение, он имеет вид

return выражение

Здесь тип выражения должен совпадать с типом возвращаемого значения, указанном в заголовке функции.

Для функции, не возвращающей значения, оператор завершения не содержит выражения. Кроме того, такая функция вообще может обойтись без оператора завершения, потому что её выполнение будет автоматически прекращено после выполнения последнего оператора в её теле.

3.4.8 Оператор-предупреждение

Оператор-предупреждение обеспечивает аварийное завершение программы при невыполнении некоторого условия:

assert условие

Например, оператор

```
assert x > 0
```

вызовет завершение программы, если значение x меньше или равно 0.